

Code Explanation

Workflow Explanation: E2E AC TXDBH DP YUYU Creation

The provided code is a Python script that implements a workflow for E2E AC TXDBH DP YUYU Creation using Apache Spark. The workflow involves reading data from Oracle tables, applying transformations, and writing the processed data to output files and staging tables.

Overview of the Code

The code is organized into several functions, each representing a specific step in the workflow. The main function, `run_workflow`, orchestrates the entire process by calling other functions in a specific order. The workflow involves checking source records, processing data, and creating trigger files.

Step 1: Building Source SQL Query

This step involves constructing a SQL query to extract data from the source Oracle table. The query is built using the `build_source_sql` function, which takes a `WorkflowConfig` object as input.

```
def build_source_sql(config: WorkflowConfig) -> str:
    return f"""
    SELECT
        t.T_TX_REQUEST_REQUEST_ID,
        t.T_TX_BASIC_TRANS_ID,
        t.T_TX_RELATION_TRANS_REL_ID,
        t.T_TX_REQ_POL_REQUEST_POLICY_ID,
        t.STG_POLICY_ID,
        t.T_TX_REQUEST_ORIGIN_REQUEST_ID AS ORIGIN_REQUEST_ID,
        t.T_TX_BASIC_POLICY_ID AS POLICY_ID,
        t.T_TX_REQUEST_REQ_ACC_DATETIME,
        t.T_TX_DTL_ADD_CH_N_TRANS_ZIP,
        t.T_TX_DTL_ADD_CH_N_TRANS_ADD1,
        t.T_TX_DTL_ADD_CH_N_TRANS_ADD2,
        t.T_TX_DTL_ADD_CH_N_TRANS_ADD3,
        t.T_TX_DTL_ADD_CH_N_TRANS_AD1_KJ,
        t.T_TX_DTL_ADD_CH_N_TRANS_AD2_KJ,
        t.T_TX_DTL_ADD_CH_N_TRANS_AD3_KJ,
        t.T_TX_DTL_ADD_CH_N_TRANS_PHNO,
        t.STG_TXDB_STATUS,
        t.PPAY_ZIP,
        t.PPAY_ADR1_FW,
        t.PPAY_ADR2_FW,
        t.PPAY_ADR3_FW
    FROM
        {config.stg_schema}.{config.stg_table} t
    WHERE
        t.T_TX_REQUEST_REQUEST_STATUS = 'ACTIVE'
    """
```

Step 2: Checking Source Records

This step involves checking the count of source records and writing the count to a file. The `check_source_records` function reads the source data using the SQL query built in Step 1 and counts the records.

```
def check_source_records(spark: SparkSession, config: WorkflowConfig) -> int:
    source_sql = build_source_sql(config)
    source_df = read_oracle_table(spark, config.db_connection_stg, None, None, query=source_sql)
    count = check_source_record_count(source_df)
    write_count_file(spark, count, config.output_dir, config.output_file_cnt_chk)
    return count
```

Step 3: Processing YUYU Creation

This step involves processing the data for YUYU creation. The `process\_yuyu\_creation` function reads the source data, applies transformations, and writes the processed data to output files and staging tables.

```
def process_yuyu_creation(spark: SparkSession, config: WorkflowConfig) -> None:
    source_sql = build_source_sql(config)
    source_df = read_oracle_table(spark, config.db_connection_stg, None, None, query=source_sql)
    # ... (rest of the code remains the same)
```

Step 4: Running the Workflow

This step involves running the entire workflow. The `run\_workflow` function checks the source records, determines the workflow path based on the source count, and calls the `process\_yuyu\_creation` function if there are records to process.

```
def run_workflow(spark: SparkSession, config: WorkflowConfig) -> None:
    source_count = check_source_records(spark, config)
    if source_count == 0:
        create_trigger_file(config.output_dir, config.trigger_file)
        run_cleanup_script(os.environ.get("WF_SHELL_DIR", "/scripts"), "E2E_AC_DelFile_DPAddressChangeYUYU.sh")
    else:
        process_yuyu_creation(spark, config)
        create_trigger_file(config.output_dir, config.trigger_file, "1")
        run_cleanup_script(os.environ.get("WF_SHELL_DIR", "/scripts"), "E2E AC DelFile DPAddressChangeYUYU.sh")
```